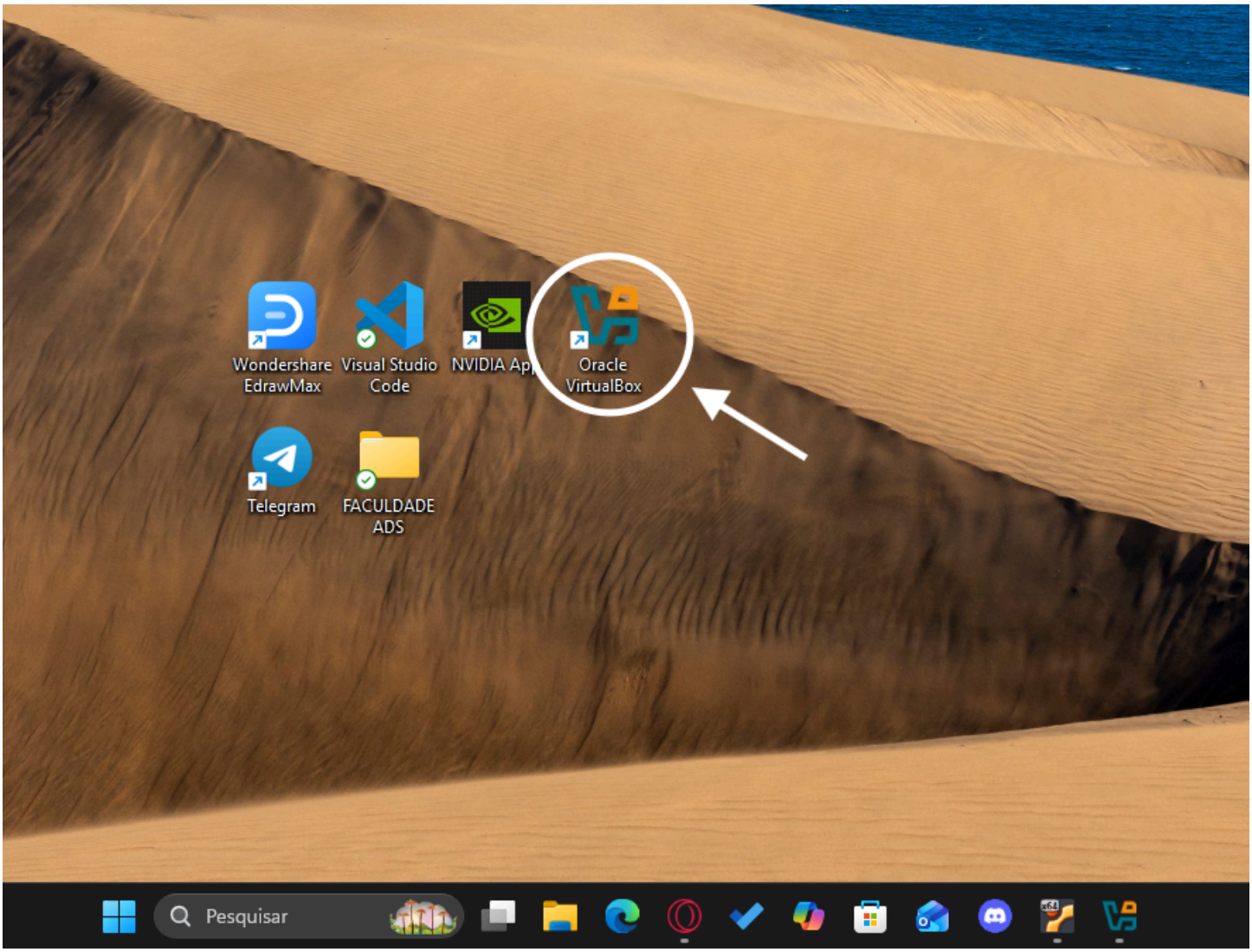
Evidencias do aDiagnóstico de Processos em Linux

Para esse projeto, utilizamos uma máquina virtual rodando o Ubuntu 25.04. Para realizar a atividade, escolhemos o jogo Minecraft, com o objetivo de analisar os processos executados pelo sistema. Durante a análise, identificamos dois processos principais, sendo um responsável pelo launcher, onde fica a tela inicial antes de entrar no jogo, e outro responsável pela execução do jogo em si, que é onde começamos a jogar. Existem outros processos relacionados ao Minecraft, porém, para facilitar a análise, filtramos e apresentamos apenas esses dois principais.

Preparando o Ambiente
Nesta etapa, vamos apresentar o sistema operacional e a aplicação utilizada na atividade, mostrando também como cada uma delas foi iniciada e preparada para a execução da atividade.

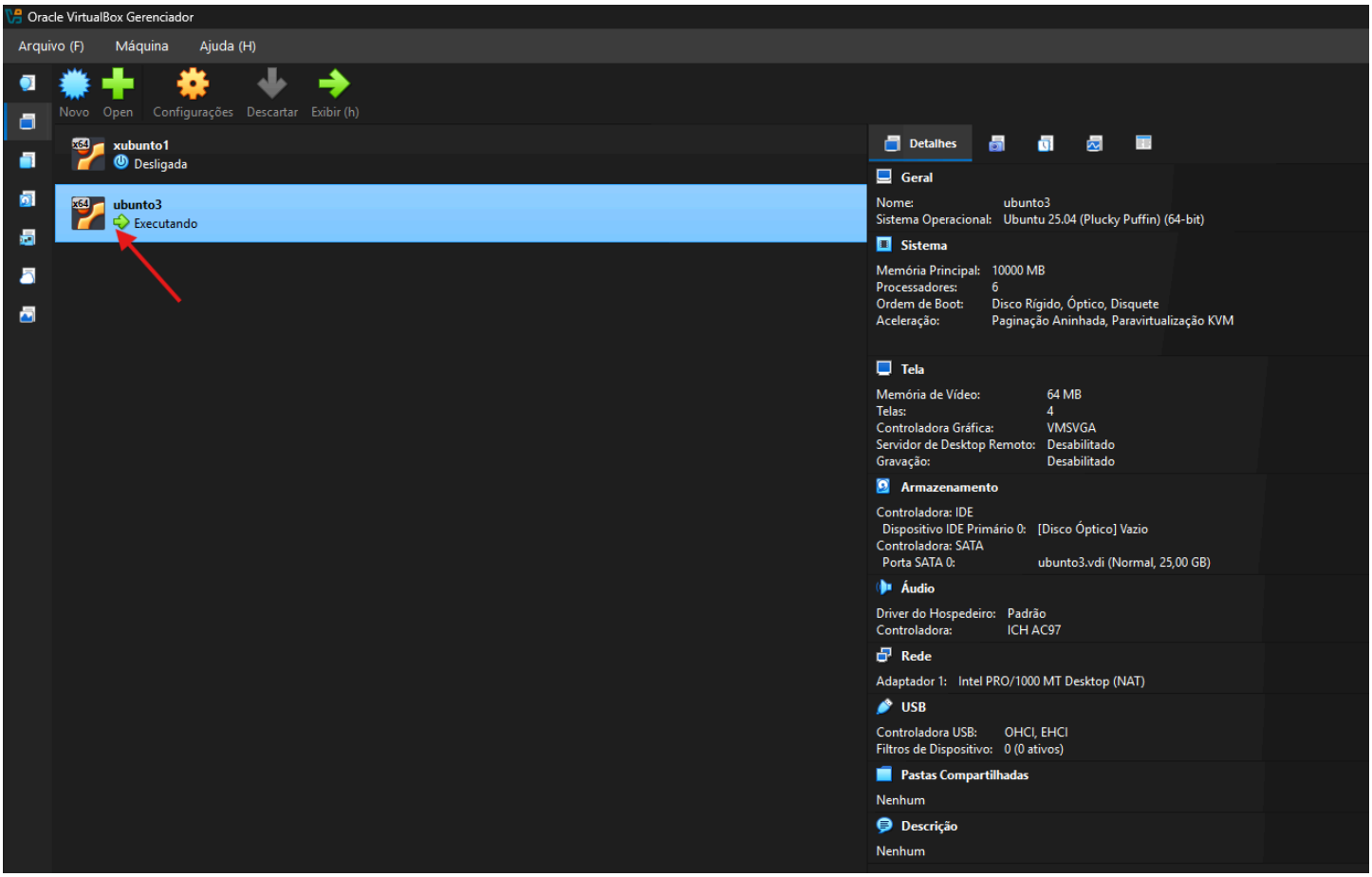
Preparando o Ambiente

- abrindo a maquina virtual



vamos executar o programa oracle VirtualboX clicando duas vezes em cima assim como na imagem .

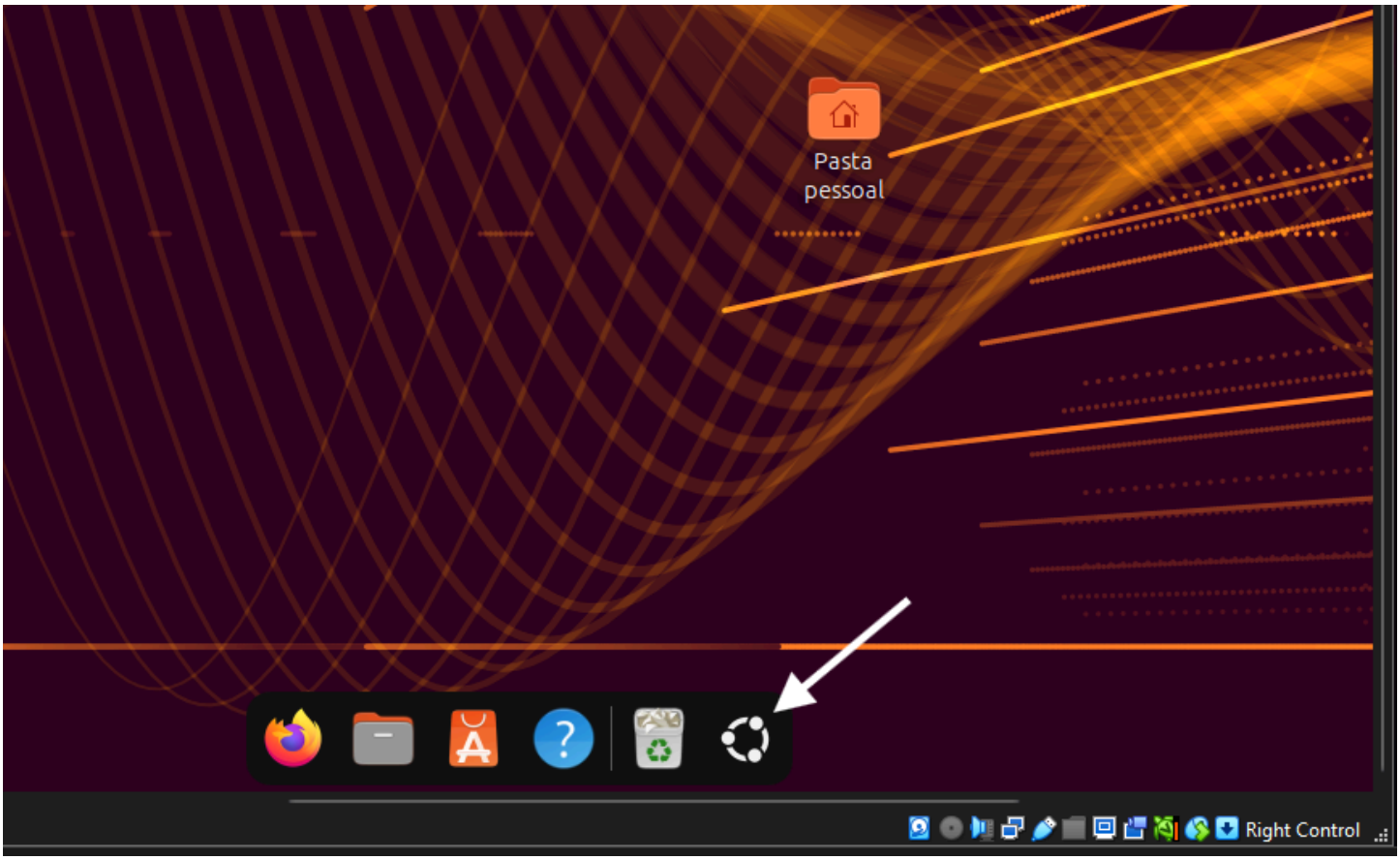
- Iniciando a maquina virtual com o com o linux (ubunto 25.04)



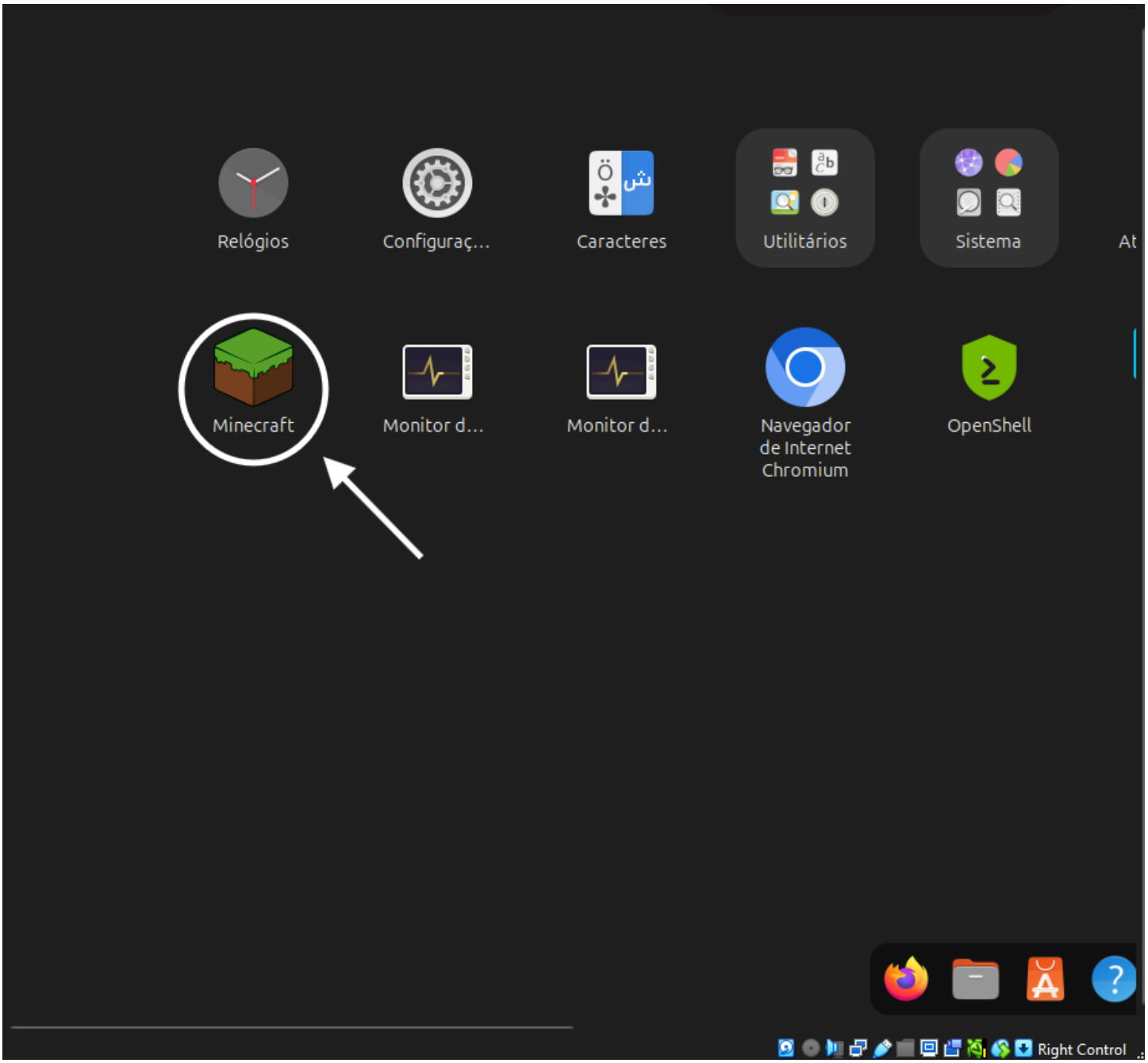
Dentro da maquina virtual vamos executar o sistema linux (ubunto) uma obs e que esse sistema virtual nao vem instalado na maquina então você vai ter que baixar a ISO de sua preferencia ubunto ou xubunto sendo sistema operacional linux e o que importa .

Dentro do sistema / executando a aplicação.

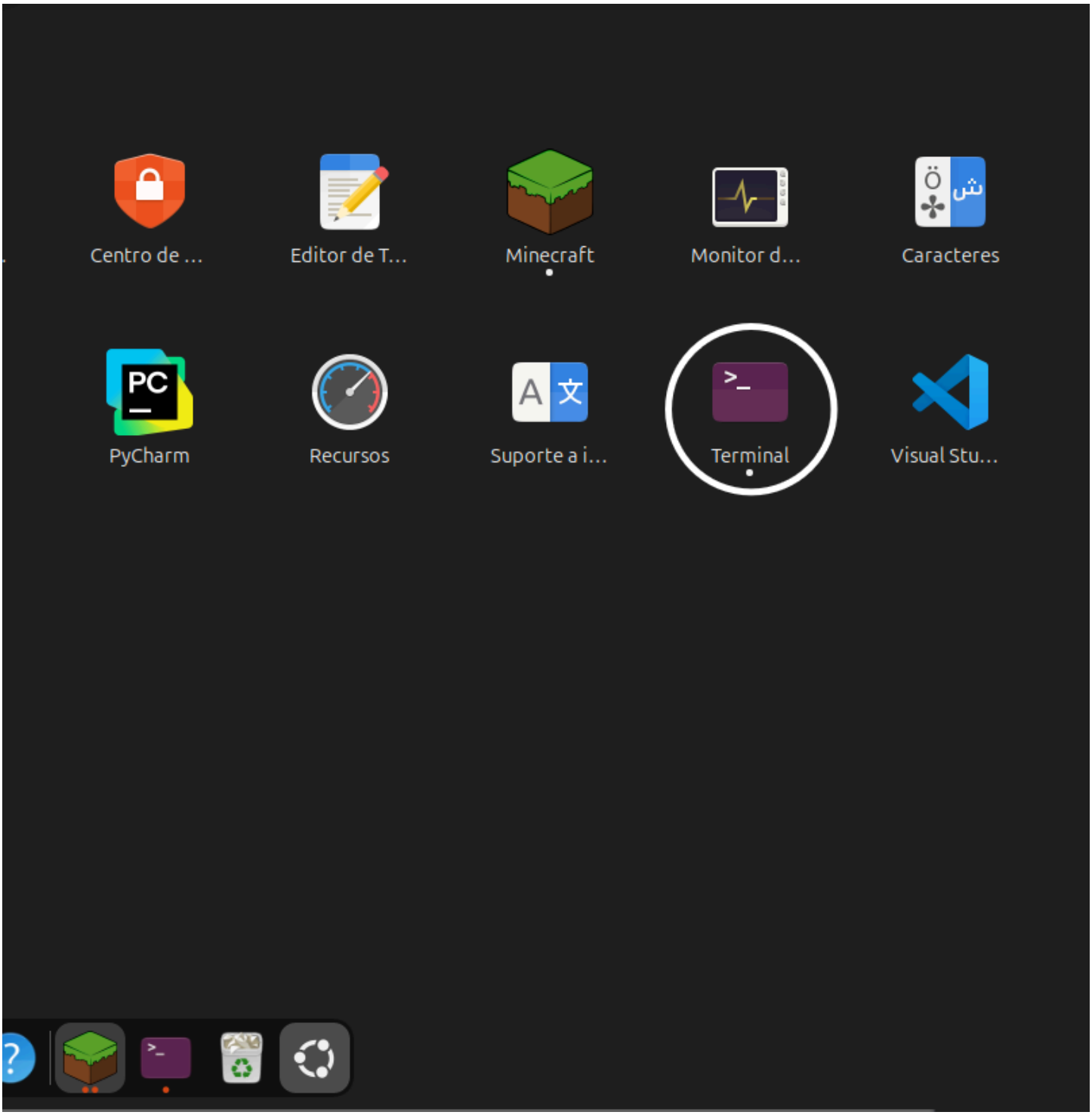
vamos em “mostra aplicativos”



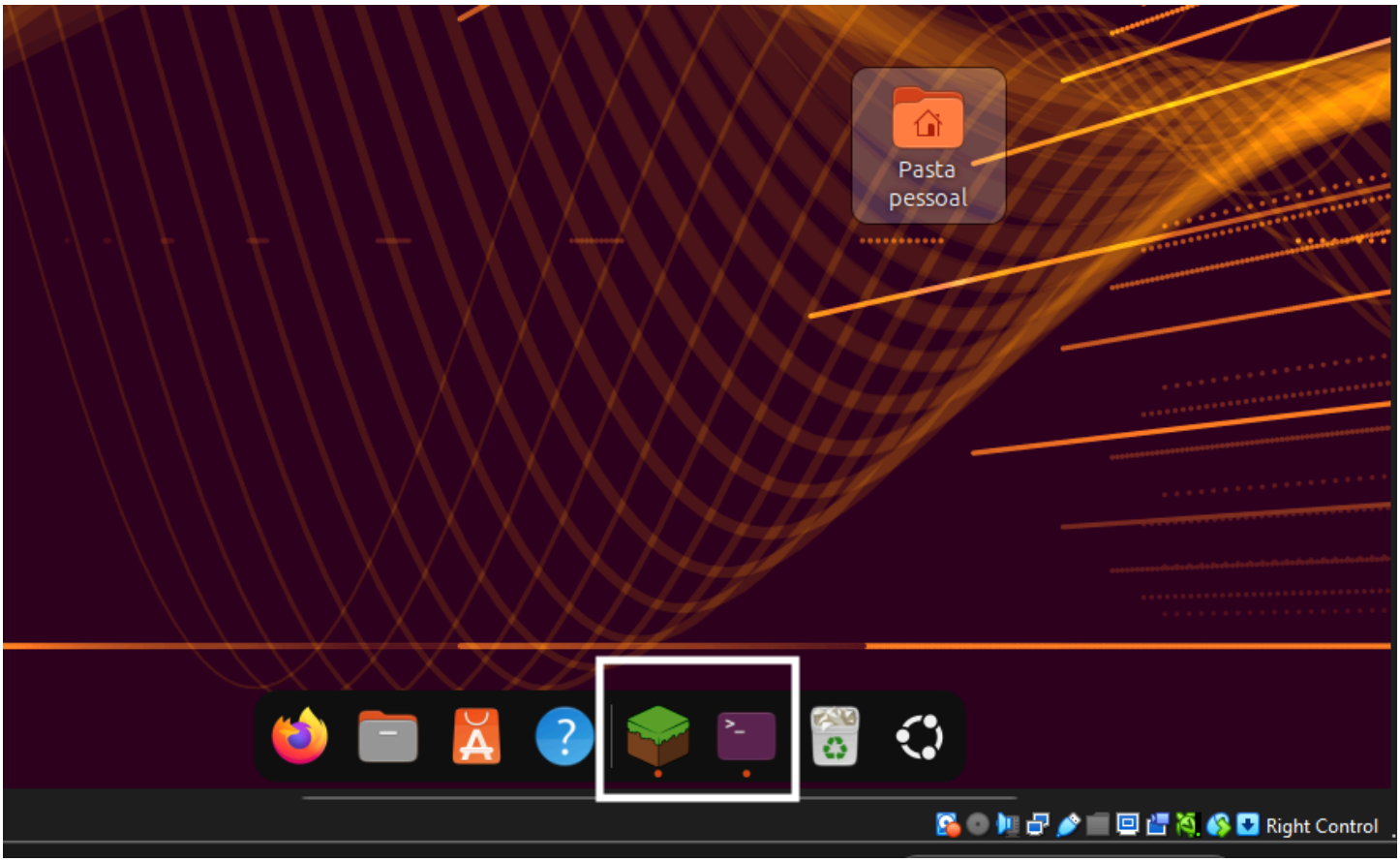
Dentro do “mostra aplicativos” vamos clicar na aplicação escolhida para atividade (minecraft)



Vamos abri tambem o terminal onde vamos fazer o diaginotico de processos

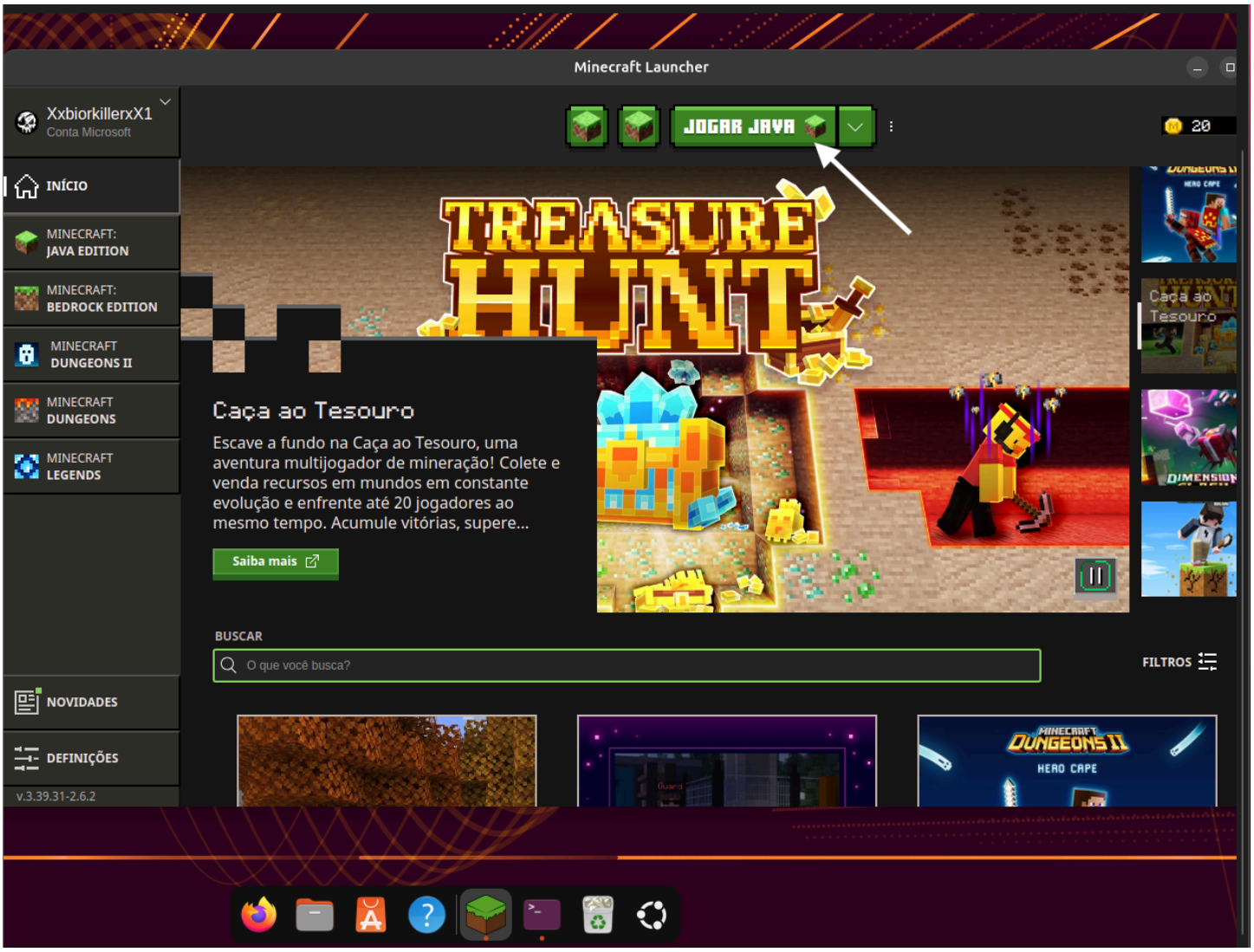


vamos ter que esta com esse dois abertos para poder fazer a atividade.

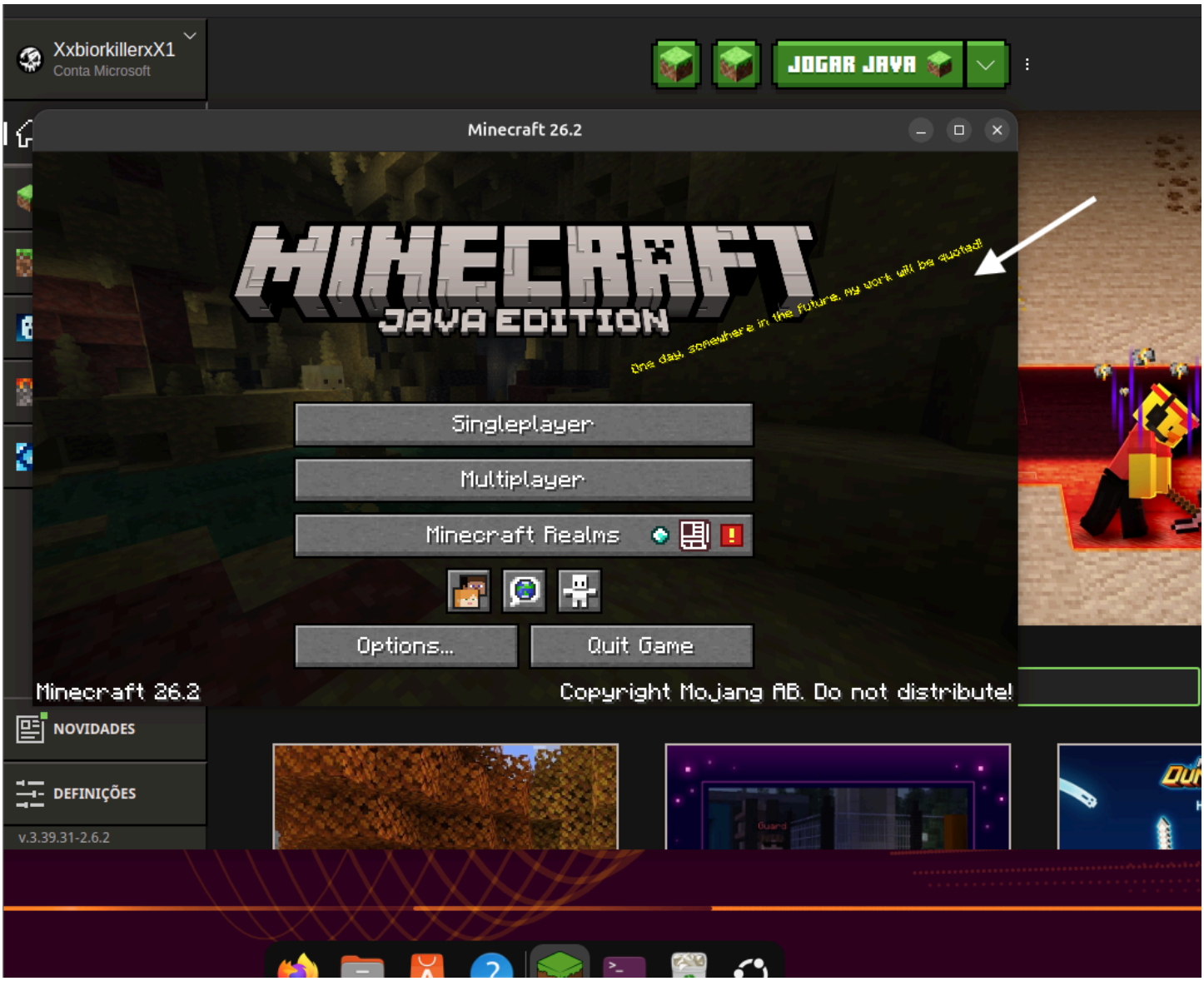


Com o minecraft aberto vamos clicar em jogar java assim como na imagem 1 e aguardar a segunda janela que e de fato o jogo assim como na imagem 2.

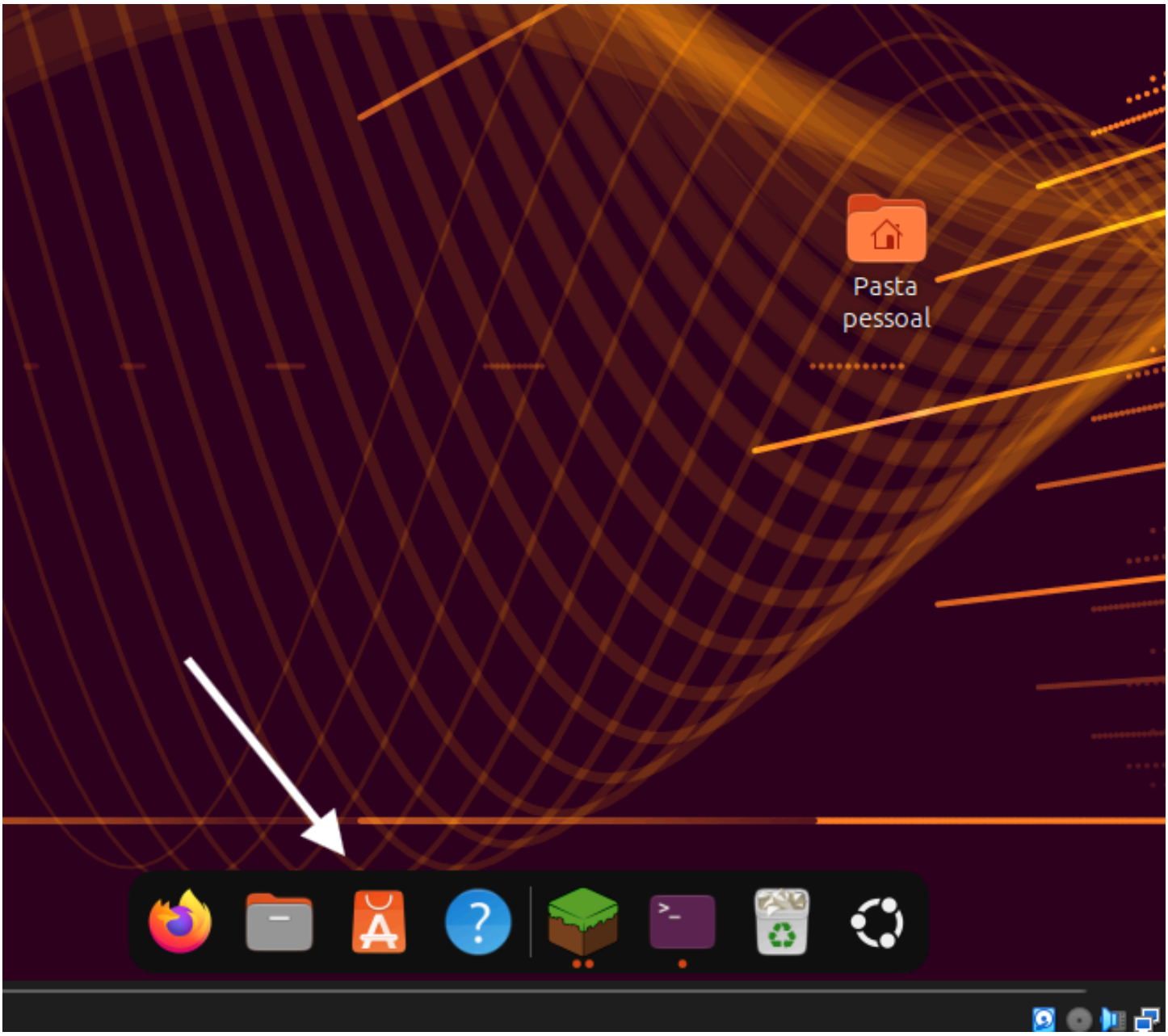
imagem(1)


Minecraft Launcher
XxbiorkillerxX1
Conta Microsoft
JOGAR JAVA
20
INÍCIO
MINECRAFT: JAVA EDITION
MINECRAFT: BEDROCK EDITION
MINECRAFT DUNGEONS II
MINECRAFT DUNGEONS
MINECRAFT LEGENDS
TREASURE HUNT
Caça ao Tesouro
Escave a fundo na Caça ao Tesouro, uma aventura multijogador de mineração! Colete e venda recursos em mundos em constante evolução e enfrente até 20 jogadores ao mesmo tempo. Acumule vitórias, supere...
Saiba mais
BUSCAR
O que você busca?
FILTROS
NOVIDADES
DEFINIÇÕES
v.3.39.31-2.6.2
MINECRAFT DUNGEONS II
HERO CAPE

imagem(2)



Com tudo isso feito termina a parte de preparo do ambiente agora podemos prosseguir com a segunda parte que e identificar o processos pelo terminal.
OBS: o jogo não vem instalado tive que instalar para fazer essa atividade, para baixar e somente ir em “centro de aplicativos” e pesquisar pelo nome do jogo assim como na imagem abaixo.



Identificando o processo:

Aqui vamos utilizar um comando para localizar o PID e o PPID dos processos da aplicação escolhida. Com esses identificadores, podemos acompanhar informações como o uso de CPU e memória.

Identificando o processo.

- **Identificando o PID e o PPID:** No terminal, utilizamos o comando **pgrep minecraft** para identificar os processos relacionados ao Minecraft. Com ele, conseguimos localizar os PID e, a partir deles, identificar o processo pai (PPID). Na imagem abaixo, podemos visualizar o PID e PPID encontrados o primeiro PID e o PPID e os demais abaixo são os filhos.

```
vitorbeckham@ubunto3:~$ pgrep minecraft
5007
5009
5197
5198
5219
5222
5228
5256
5260
5328
vitorbeckham@ubunto3:~$
```

Mapeando e definindo a hierarquia do processos
Vamos utilizar o comando **pstree** para visualizar a árvore de processos. Dessa forma, conseguimos identificar e mapear a hierarquia entre os processos, entendendo qual processo é o pai e quais são seus processos filhos.

Mapear/definir hierarquia

No terminal, após conseguir o PID e o PPID, vamos utilizar outro comando, o **pstree**. Ele é responsável por mostrar a árvore de processos, permitindo visualizar a hierarquia e identificar quais processos iremos analisar.

Para utilizar o comando, colocamos o **-p** junto do primeiro PID encontrado pelo comando pgrep minecraft. No nosso exemplo, utilizamos:

pstree -p 5007

Como podemos ver na imagem abaixo, o comando mostrou a árvore completa desse processo. Para esta atividade, iremos analisar dois processos principais: o **PPID 5009** e o **PID 5378**.

O **PPID 5009** corresponde ao launcher, que é por onde conseguimos iniciar o jogo. Já o **PID 5378** corresponde ao processo do jogo em si, onde de fato podemos jogar e interagir com o Minecraft.

```
vitorbeckham@ubunto3:~$ pstree -p 5007
minecraft-launc(5007)─┬─minecraft-launc(5009)─┬─java(5378)─┬─{java}(5380)
                      │                       │            ├─{java}(5381)
                      │                       │            ├─{java}(5382)
                      │                       │            ├─{java}(5383)
                      │                       │            ├─{java}(5384)
                      │                       │            ├─{java}(5385)
                      │                       │            ├─{java}(5386)
                      │                       │            ├─{java}(5387)
                      │                       │            ├─{java}(5388)
                      │                       │            ├─{java}(5389)
                      │                       │            ├─{java}(5390)
                      │                       │            ├─{java}(5391)
                      │                       │            ├─{java}(5392)
                      │                       │            ├─{java}(5393)
                      │                       │            ├─{java}(5394)
                      │                       │            ├─{java}(5395)
                      │                       │            ├─{java}(5396)
                      │                       │            ├─{java}(5397)
                      │                       │            ├─{java}(5398)
                      │                       │            ├─{java}(5399)
                      │                       │            ├─{java}(5400)
                      │                       │            ├─{java}(5401)
                      │                       │            ├─{java}(5402)
                      │                       │            ├─{java}(5403)
                      │                       │            ├─{java}(5404)
                      │                       │            ├─{java}(5405)
                      │                       │            ├─{java}(5412)
                      │                       │            ├─{java}(5423)
                      │                       │            ├─{java}(5424)
                      │                       │            ├─{java}(5425)
                      │                       │            ├─{java}(5427)
                      │                       │            ├─{java}(5428)
                      │                       │            ├─{java}(5429)
                      │                       │            ├─{java}(5430)
                      │                       │            ├─{java}(5431)
                      │                       │            ├─{java}(5432)
                      │                       │            ├─{java}(5433)
                      │                       │            ├─{java}(5434)
                      │                       │            ├─{java}(5435)
                      │                       │            ├─{java}(5436)
                      │                       │            ├─{java}(5437)
                      │                       │            ├─{java}(5438)
                      │                       │            ├─{java}(5439)
                      │                       │            ├─{java}(5443)
                      │                       │            ├─{java}(5445)
                      │                       │            ├─{java}(5456)
                      │                       │            ├─{java}(5458)
                      │                       │            ├─{java}(5459)
                      │                       │            └─{java}(6164)
                      │                       ├─minecraft-launc(5197)──minecraft-launc(5219)─┬─{minecraft-launc}(5300)
                      │                       │                                              ├─{minecraft-launc}(5301)
                      │                       │                                              ├─{minecraft-launc}(5302)
                      │                       │                                              ├─{minecraft-launc}(5303)
                      │                       │                                              ├─{minecraft-launc}(5304)
                      │                       │                                              ├─{minecraft-launc}(5305)
                      │                       │                                              ├─{minecraft-launc}(5306)
                      │                       │                                              ├─{minecraft-launc}(5307)
                      │                       │                                              ├─{minecraft-launc}(5309)
                      │                       │                                              ├─{minecraft-launc}(5310)
                      │                       │                                              ├─{minecraft-launc}(5311)
                      │                       │                                              ├─{minecraft-launc}(5312)
                      │                       │                                              ├─{minecraft-launc}(5313)
                      │                       │                                              ├─{minecraft-launc}(5314)
                      │                       │                                              └─{minecraft-launc}(5322)
```

```
│                       ├─minecraft-launc(5198)─┬─minecraft-launc(5228)─┬─{minecraft-launc}(5229)
│                       │                       │                       ├─{minecraft-launc}(5230)
│                       │                       │                       ├─{minecraft-launc}(5231)
│                       │                       │                       ├─{minecraft-launc}(5232)
│                       │                       │                       ├─{minecraft-launc}(5233)
│                       │                       │                       └─{minecraft-launc}(5323)
│                       │                       ├─minecraft-launc(5256)─┬─{minecraft-launc}(5259)
│                       │                       │                       ├─{minecraft-launc}(5274)
│                       │                       │                       ├─{minecraft-launc}(5276)
│                       │                       │                       ├─{minecraft-launc}(5277)
│                       │                       │                       ├─{minecraft-launc}(5278)
│                       │                       │                       ├─{minecraft-launc}(5283)
│                       │                       │                       ├─{minecraft-launc}(5284)
│                       │                       │                       ├─{minecraft-launc}(5296)
│                       │                       │                       ├─{minecraft-launc}(5297)
│                       │                       │                       └─{minecraft-launc}(5324)
│                       │                       └─minecraft-launc(5260)─┬─{minecraft-launc}(5269)
│                       │                                               ├─{minecraft-launc}(5279)
│                       │                                               ├─{minecraft-launc}(5280)
│                       │                                               ├─{minecraft-launc}(5281)
│                       │                                               ├─{minecraft-launc}(5282)
│                       │                                               ├─{minecraft-launc}(5285)
│                       │                                               ├─{minecraft-launc}(5298)
│                       │                                               ├─{minecraft-launc}(5299)
│                       │                                               ├─{minecraft-launc}(5317)
│                       │                                               ├─{minecraft-launc}(5318)
│                       │                                               ├─{minecraft-launc}(5319)
│                       │                                               ├─{minecraft-launc}(5326)
│                       │                                               ├─{minecraft-launc}(5335)
│                       │                                               ├─{minecraft-launc}(5337)
│                       │                                               ├─{minecraft-launc}(5338)
│                       │                                               └─{minecraft-launc}(5339)
│                       ├─minecraft-launc(5222)─┬─{minecraft-launc}(5261)
│                       │                       ├─{minecraft-launc}(5262)
│                       │                       ├─{minecraft-launc}(5263)
│                       │                       ├─{minecraft-launc}(5264)
│                       │                       ├─{minecraft-launc}(5265)
│                       │                       ├─{minecraft-launc}(5266)
│                       │                       ├─{minecraft-launc}(5267)
│                       │                       ├─{minecraft-launc}(5268)
│                       │                       ├─{minecraft-launc}(5291)
│                       │                       └─{minecraft-launc}(5321)
│                       ├─minecraft-launc(5328)─┬─{minecraft-launc}(5329)
│                       │                       ├─{minecraft-launc}(5330)
│                       │                       ├─{minecraft-launc}(5331)
│                       │                       ├─{minecraft-launc}(5332)
│                       │                       ├─{minecraft-launc}(5333)
│                       │                       └─{minecraft-launc}(5334)
│                       ├─{minecraft-launc}(5010)
│                       ├─{minecraft-launc}(5012)
│                       ├─{minecraft-launc}(5013)
│                       ├─{minecraft-launc}(5014)
│                       ├─{minecraft-launc}(5097)
│                       ├─{minecraft-launc}(5181)
│                       ├─{minecraft-launc}(5182)
│                       ├─{minecraft-launc}(5183)
│                       ├─{minecraft-launc}(5184)
│                       ├─{minecraft-launc}(5186)
│                       ├─{minecraft-launc}(5187)
│                       ├─{minecraft-launc}(5189)
│                       ├─{minecraft-launc}(5191)
│                       ├─{minecraft-launc}(5192)
│                       ├─{minecraft-launc}(5193)
│                       ├─{minecraft-launc}(5194)
│                       ├─{minecraft-launc}(5196)
│                       ├─{minecraft-launc}(5199)
```

```
│                        ├─{minecraft-launc}(5199)
│                        ├─{minecraft-launc}(5200)
│                        ├─{minecraft-launc}(5201)
│                        ├─{minecraft-launc}(5202)
│                        ├─{minecraft-launc}(5203)
│                        ├─{minecraft-launc}(5204)
│                        ├─{minecraft-launc}(5205)
│                        ├─{minecraft-launc}(5206)
│                        ├─{minecraft-launc}(5207)
│                        ├─{minecraft-launc}(5208)
│                        ├─{minecraft-launc}(5209)
│                        ├─{minecraft-launc}(5210)
│                        ├─{minecraft-launc}(5211)
│                        ├─{minecraft-launc}(5212)
│                        ├─{minecraft-launc}(5213)
│                        ├─{minecraft-launc}(5214)
│                        ├─{minecraft-launc}(5215)
│                        ├─{minecraft-launc}(5216)
│                        ├─{minecraft-launc}(5217)
│                        ├─{minecraft-launc}(5218)
│                        ├─{minecraft-launc}(5234)
│                        ├─{minecraft-launc}(5235)
│                        ├─{minecraft-launc}(5236)
│                        ├─{minecraft-launc}(5237)
│                        ├─{minecraft-launc}(5238)
│                        ├─{minecraft-launc}(5242)
│                        ├─{minecraft-launc}(5243)
│                        ├─{minecraft-launc}(5244)
│                        ├─{minecraft-launc}(5245)
│                        ├─{minecraft-launc}(5250)
│                        ├─{minecraft-launc}(5295)
│                        ├─{minecraft-launc}(5327)
│                        └─{minecraft-launc}(5379)
└─{minecraft-launc}(5008)
```

Registrando informações do processo:

Antes de fazer qualquer alteração no processo, vamos registrar algumas informações importantes, como o estado, uso de CPU, memória, prioridade e valor de nice. Dessa forma, teremos uma base para comparar o processo antes e depois das alterações.

Coletando o estado inicial

Obtendo o estado inicial do processo:

Para obter o estado inicial dos processos que vamos analisar, vamos utilizar o comando **ps -o stat,pri,ni,%cpu,%mem,cmd -p + PID**. Neste caso, iremos analisar dois processos: o **PID 5009** e o **PID 5378**.

Nosso comando ficará assim:

```
ps -o star,pri,ni,%cpu,%mem,cmd -p 5043
```

```
vitorbeckham@ubunto3:~$ ps -o stat,pri,ni,%cpu,%mem,cmd -p 5009
STAT PRI  NI %CPU %MEM CMD
SLl   19   0  2.6  4.7 /home/vitorbeckham/snap/mc-installer/646/.minecraft/launcher/minecraft-launcher --chainLoad
```

ps -o star,pri,ni,%cpu,%mem,cmd -p 5403

```
vitorbeckham@ubunto3:~$ ps -o stat,pri,ni,%cpu,%mem,cmd -p 5378
STAT PRI  NI %CPU %MEM CMD
Sl    19   0 46.5 31.4 /home/vitorbeckham/snap/mc-installer/646/.minecraft/runtime/java-runtime-epsilon/linux/java-runtime-epsilon/bin/
```

Como podemos observar nas evidências acima, o processo **5009** está com prioridade **19**, sem valor de nice informado. O consumo de CPU está em **2,6%** e o de memória em **4,7%**.

Já no segundo processo, o **5378**, também temos prioridade **19** e nenhum valor de nice informado. A diferença está no consumo de recursos: ele está utilizando **46,5% de CPU** e **31,4% de memória**.

Verificando as threads:

Agora vamos verificar as threads utilizadas pelos processos que estamos analisando. Com isso, podemos identificar como a aplicação está trabalhando e quantas threads estão sendo utilizadas durante sua execução.

Threads

Aqui vamos entender o que são threads e como podemos identificá-las. As threads são unidades de execução que fazem parte de um processo e podem ser responsáveis por diferentes tarefas dentro da aplicação.

Para visualizar as threads de um processo, vamos utilizar o comando **ps -L -p + PID**. No nosso caso, vamos executar o comando para os dois PID que estamos analisando, como podemos ver abaixo.

ps -L -p 5009

```
vitorbeckham@ubunto3:~$ ps -L -p 5009
    PID     LWP TTY          TIME CMD
   5009    5009 ?        00:00:04 minecraft-launc
   5009    5010 ?        00:00:00 minecraft-launc
   5009    5012 ?        00:00:00 pool-spawner
   5009    5013 ?        00:00:00 gmain
   5009    5014 ?        00:00:00 gdbus
   5009    5097 ?        00:00:00 minecraft-launc
   5009    5181 ?        00:00:03 minecraft-launc
   5009    5182 ?        00:00:00 minecraft-launc
   5009    5183 ?        00:00:09 minecraft-launc
   5009    5184 ?        00:00:00 minecraft-launc
   5009    5186 ?        00:00:00 minecraft-launc
   5009    5187 ?        00:00:00 minecraft-launc
   5009    5189 ?        00:00:00 minecraft-launc
   5009    5191 ?        00:00:03 minecraft-launc
   5009    5192 ?        00:00:02 minecraft-launc
   5009    5193 ?        00:00:00 minecraft-launc
   5009    5194 ?        00:00:00 minecraft-launc
   5009    5196 ?        00:00:00 sandbox_ipc_thr
   5009    5199 ?        00:00:00 HangWatcher
   5009    5200 ?        00:00:00 ThreadPoolServi
   5009    5201 ?        00:00:00 ThreadPoolForeg
   5009    5202 ?        00:00:01 ThreadPoolSingl
   5009    5203 ?        00:00:00 ThreadPoolForeg
   5009    5204 ?        00:00:00 ThreadPoolForeg
   5009    5205 ?        00:00:02 Chrome_IOThread
   5009    5206 ?        00:00:00 MemoryInfra
   5009    5207 ?        00:00:00 [pango] fontcon
   5009    5208 ?        00:00:00 Bluez D-Bus thr
   5009    5209 ?        00:00:00 CrShutdownDetec
   5009    5210 ?        00:00:00 dconf worker
   5009    5211 ?        00:00:00 ThreadPoolForeg
   5009    5212 ?        00:00:00 ThreadPoolForeg
   5009    5213 ?        00:00:00 ThreadPoolForeg
   5009    5214 ?        00:00:00 ThreadPoolForeg
   5009    5215 ?        00:00:00 CompositorTileW
   5009    5216 ?        00:00:00 inotify_reader
   5009    5217 ?        00:00:00 ThreadPoolSingl
   5009    5218 ?        00:00:00 VideoCaptureThr
   5009    5234 ?        00:00:00 ThreadPoolSingl
   5009    5235 ?        00:00:00 ThreadPoolForeg
   5009    5236 ?        00:00:00 ThreadPoolForeg
   5009    5237 ?        00:00:00 ThreadPoolForeg
   5009    5238 ?        00:00:00 ThreadPoolForeg
   5009    5242 ?        00:00:00 ThreadPoolForeg
   5009    5243 ?        00:00:00 ThreadPoolForeg
   5009    5244 ?        00:00:00 ThreadPoolForeg
   5009    5245 ?        00:00:00 ThreadPoolForeg
   5009    5250 ?        00:00:00 ThreadPoolForeg
   5009    5295 ?        00:00:00 BatteryStatusNo
   5009    5327 ?        00:00:40 Gamepad polling
   5009    5379 ?        00:00:00 minecraft-launc
vitorbeckham@ubunto3:~$
```

ps -L -p 5378

```
vitorbeckham@ubunto3:~$ ps -L -p 5378
    PID     LWP TTY          TIME CMD
   5378    5378 ?        00:00:00 java
   5378    5380 ?        00:05:55 Render thread
   5378    5381 ?        00:00:00 ZUncommitter#0
   5378    5382 ?        00:00:03 ZWorkerOld#0
   5378    5383 ?        00:00:04 ZWorkerOld#1
   5378    5384 ?        00:00:01 ZWorkerYoung#0
   5378    5385 ?        00:00:01 ZWorkerYoung#1
   5378    5386 ?        00:00:00 ZDriverMinor
   5378    5387 ?        00:00:00 ZDriverMajor
   5378    5388 ?        00:00:20 ZDirector
   5378    5389 ?        00:00:00 ZStat
   5378    5390 ?        00:00:00 RuntimeWorker#0
   5378    5391 ?        00:00:00 RuntimeWorker#1
   5378    5392 ?        00:00:00 RuntimeWorker#2
   5378    5393 ?        00:00:00 RuntimeWorker#3
   5378    5394 ?        00:00:00 VM Periodic Tas
   5378    5395 ?        00:00:00 VM Thread
   5378    5396 ?        00:00:00 Reference Handl
   5378    5397 ?        00:00:00 Finalizer
   5378    5398 ?        00:00:00 Signal Dispatch
   5378    5399 ?        00:00:00 Service Thread
   5378    5400 ?        00:00:00 Monitor Deflati
   5378    5401 ?        00:00:16 C2 CompilerThre
   5378    5402 ?        00:00:04 C1 CompilerThre
   5378    5403 ?        00:00:00 StringDedupThre
   5378    5404 ?        00:00:00 Notification Th
   5378    5405 ?        00:00:00 Common-Cleaner
   5378    5412 ?        00:00:00 JNA Cleaner
   5378    5423 ?        00:00:00 Timer hack thre
   5378    5424 ?        00:00:00 Yggdrasil Key F
   5378    5427 ?        00:07:49 llvmpipe-0
   5378    5428 ?        00:07:56 llvmpipe-1
   5378    5429 ?        00:08:03 llvmpipe-2
   5378    5430 ?        00:08:15 llvmpipe-3
   5378    5431 ?        00:08:29 llvmpipe-4
   5378    5432 ?        00:08:57 llvmpipe-5
   5378    5433 ?        00:00:00 Render thread
   5378    5434 ?        00:00:00 Render thread
   5378    5435 ?        00:00:00 Render thread
   5378    5436 ?        00:00:00 Render thread
   5378    5437 ?        00:00:00 Render thread
   5378    5438 ?        00:00:00 Render thread
   5378    5439 ?        00:00:00 java:disk$0
   5378    5443 ?        00:00:11 Sound engine
   5378    5445 ?        00:00:00 threaded-ml
   5378    5456 ?        00:00:00 Telemetry-Sende
   5378    5458 ?        00:00:39 threaded-ml
   5378    5459 ?        00:00:00 Render thread
vitorbeckham@ubunto3:~$
```

Explorando o diretório:

Agora vamos explorar o diretório **/proc/PID** para obter informações importantes sobre os processos. Vamos analisar alguns arquivos e diretórios e verificar como essas informações podem ajudar no diagnóstico do processo.

Explorando o diretório

Aqui vamos analisar de fato o diretório relacionado aos nossos processos. Por meio do **/proc/PID**, conseguimos acessar informações importantes, como o estado do processo, a quantidade de threads utilizadas, os grupos aos quais ele pertence e informações relacionadas ao uso de memória.

Para acessar essas informações, vamos utilizar o comando **cat /proc/PID/status**. Com os dados apresentados, podemos entender melhor como o processo está funcionando e quais recursos ele está utilizando, ajudando no diagnóstico.

Na nossa atividade, vamos realizar essa análise nos dois processos escolhidos: o **PPID 5009** e o **PID 5378**.

```
cat  /proc/5009/status
```

```
vitorbeckham@ubunto3:~$ cat /proc/5009/status
Name:   minecraft-launc
Umask:  0002
State:  S (sleeping)
Tgid:   5009
Ngid:   0
Pid:    5009
PPid:   5007
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 111 114 1000
NStgid: 5009
NSpid:  5009
NSpgid: 3720
NSsid:  3720
Kthread:        0
VmPeak:  4426688 kB
VmSize:  4402260 kB
VmLck:        48 kB
VmPin:         0 kB
VmHWM:    453404 kB
VmRSS:    453404 kB
RssAnon:          203380 kB
RssFile:          248900 kB
RssShmem:           1124 kB
VmData:   653512 kB
VmStk:       136 kB
VmExe:      3156 kB
VmLib:    381352 kB
VmPTE:      1896 kB
VmSwap:        0 kB
HugetlbPages:          0 kB
CoreDumping:    0
THP_enabled:    1
untag_mask:     0xffffffffffffffff
Threads:        51
SigQ:   0/27598
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000001000
SigCgt: 0000000100014203
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000
NoNewPrivs:     0
Seccomp:        2
Seccomp_filters:        2
Speculation_Store_Bypass:       vulnerable
SpeculationIndirectBranch:      always enabled
Cpus_allowed:   3f
Cpus_allowed_list:      0-5
Mems_allowed:   00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000
0,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,0
Mems_allowed_list:      0
voluntary_ctxt_switches:        10786
nonvoluntary_ctxt_switches:     1820
x86_Thread_features:
x86_Thread_features_locked:
```

cat /proc/5378

/status

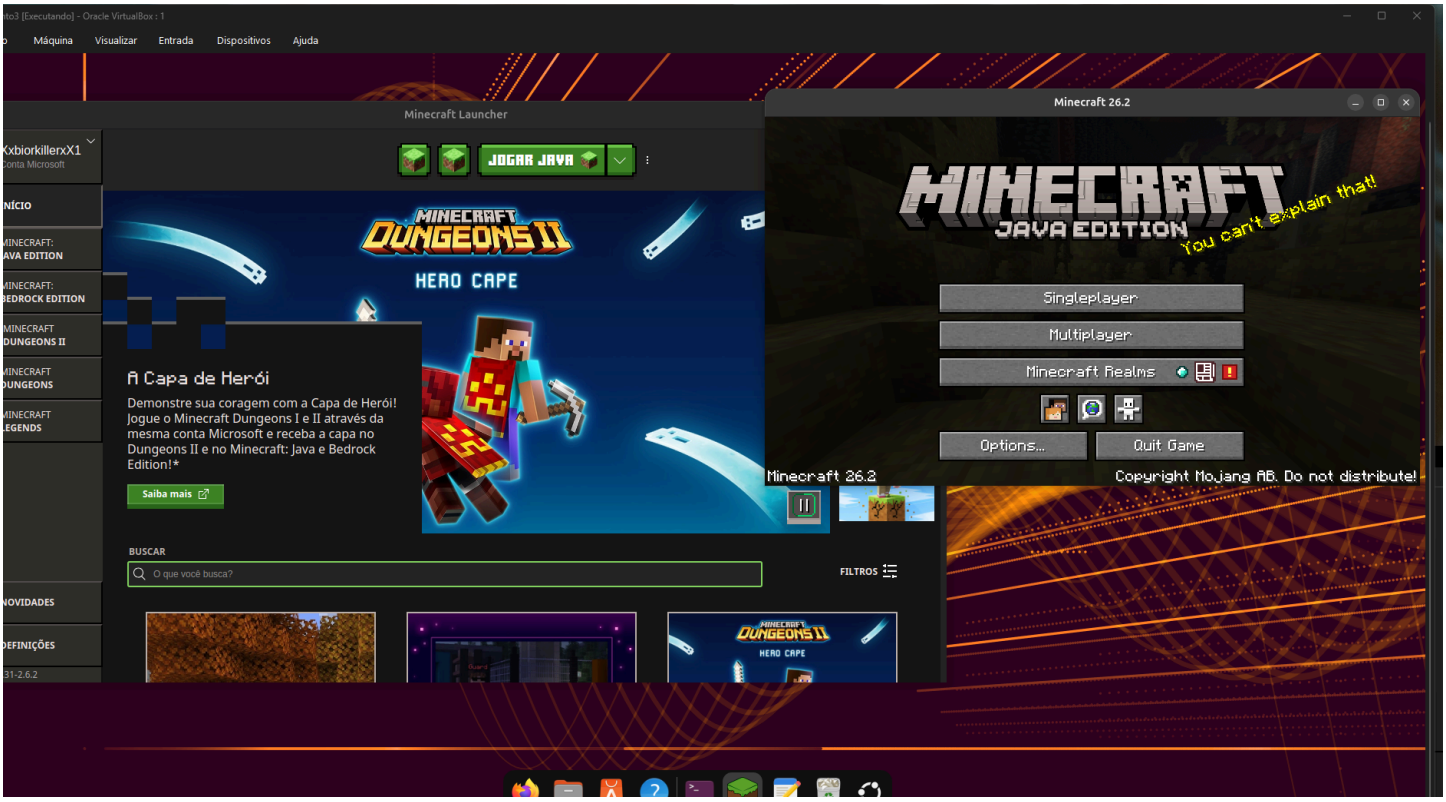
```
vitorbeckham@ubunto3:~$ cat /proc/5378/status
Name:   java
Umask:  0002
State:  S (sleeping)
Tgid:   5378
Ngid:   0
Pid:    5378
PPid:   5009
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 111 114 1000
NStgid: 5378
NSpid:  5378
NSpgid: 3720
NSsid:  3720
Kthread:        0
VmPeak: 73274008 kB
VmSize: 73274008 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:   3272952 kB
VmRSS:   3272952 kB
RssAnon:         1051984 kB
RssFile:          122088 kB
RssShmem:        2098880 kB
VmData:  1547320 kB
VmStk:       148 kB
VmExe:         4 kB
VmLib:    209896 kB
VmPTE:      7340 kB
VmSwap:        0 kB
HugetlbPages:          0 kB
CoreDumping:    0
THP_enabled:    1
untag_mask:     0xffffffffffffffff
Threads:        49
SigQ:   0/27598
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 2000000101005ccf
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000
NoNewPrivs:     0
Seccomp:        2
Seccomp_filters:        2
Speculation_Store_Bypass:       vulnerable
SpeculationIndirectBranch:      always enabled
Cpus_allowed:   3f
Cpus_allowed_list:      0-5
Mems_allowed:   00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,000000
0,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,000
Mems_allowed_list:      0
voluntary_ctxt_switches:        15
nonvoluntary_ctxt_switches:     10
x86_Thread_features:
x86_Thread_features_locked:
vitorbeckham@ubunto3:~$
```

Aplicando sinais ao processo:

Nesta etapa, vamos testar alguns sinais nos processos analisados. Primeiro, utilizaremos o **SIGSTOP** para interromper o processo e o **SIGCONT** para retomá-lo. Depois, vamos testar a alteração de prioridade e observar o comportamento do processo. Por fim, utilizaremos o **SIGTERM** para solicitar o encerramento do processo.

Aplicando sinais

Aqui vamos fazer algumas modificações nos processos e verificar o que acontece quando aplicamos diferentes sinais. Vamos observar o comportamento dos processos ao utilizar **STOP, CONT, alterar a prioridade e, por fim, o TERM**.




```
vitorbeckham@ubunto3:~$ top -p 5009

top - 03:38:23 up  1:55,  1 user,  load average: 1,50, 1,24, 1,63
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s):  5,0 us,  4,6 sy,  0,0 ni, 89,2 id,  0,1 wa,  0,0 hi,  1,1 si,  0,0 st
MB mem :   9189,4 total,    467,8 free,   6571,2 used,   4633,5 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2618,3 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5009 vitorbe+  20   0 4256460 464508 245732 S   2,0   4,9   2:17.21 minecraft-launc
```

```
vitorbeckham@ubunto3:~$ top -p 5378

top - 03:39:04 up  1:56,  1 user,  load average: 0,97, 1,14, 1,58
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s):  5,8 us,  5,1 sy,  0,0 ni, 87,9 id,  0,0 wa,  0,0 hi,  1,1 si,  0,0 st
MB mem :   9189,4 total,    434,3 free,   6602,3 used,   4665,0 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2587,1 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5378 vitorbe+  20   0   69,9g   3,1g   2,1g S  24,6  34,5 110:38.24 java
```

com o comando top -p + pid podemos ver que esta tudo funcionando normal

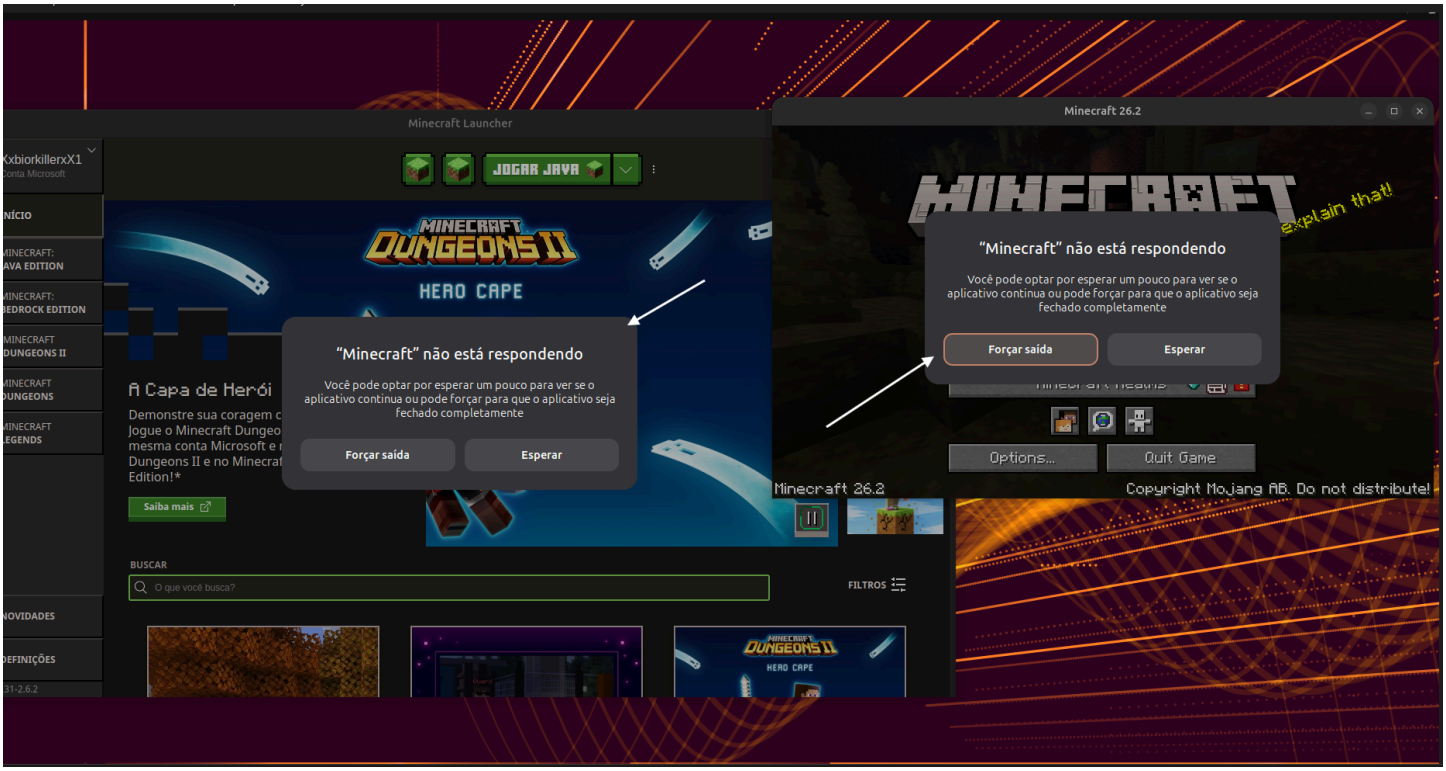
Aplicando SIGSTOP

Primeiro, vamos utilizar o comando **kill -STOP + PID** nos dois processos que estamos analisando. Esse comando serve para interromper temporariamente o processo. Como podemos observar nos exemplos abaixo, tudo que estava funcionando fica congelado, como as animações da interface, sons e cliques.

No nosso caso, utilizamos:

kill -STOP 5009
kill -STOP 5378

```
vitorbeckham@ubunto3:~$ kill -STOP 5009
vitorbeckham@ubunto3:~$ kill -STOP 5378
vitorbeckham@ubunto3:~$
```



Os dois processos mudaram de S para T, como podemos observar na imagem abaixo. Também podemos perceber que apareceu a mensagem informando que o processo não está respondendo, mostrando o efeito da aplicação do comando.

```
vitorbeckham@ubunto3:~$ top -p 5009

top - 03:33:52 up  1:51,  1 user,  load average: 0,80, 1,84, 1,95
Tarefas:   1 total,   0 em exec.,   0 dormindo,   1 parado,   0 zumbi
%CPU(s):  5,4 us,  5,3 sy,  0,0 ni, 88,5 id,  0,0 wa,  0,0 hi,  0,8 si,  0,0 st
MB mem :   9189,4 total,    400,3 free,   6643,6 used,   4755,7 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2545,8 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TEMPO+ COMANDO
   5009 vitorbe+  20   0 4387680 449664 245776 T   0,0   4,8   2:14.91 minecraft-launc
```

```
vitorbeckham@ubunto3:~$ top -p 5378

top - 03:34:18 up  1:51,  1 user,  load average: 0,81, 1,76, 1,92
Tarefas:   1 total,   0 em exec.,   0 dormindo,   1 parado,   0 zumbi
%CPU(s):  3,7 us,  3,4 sy,  0,0 ni, 91,8 id,  0,0 wa,  0,0 hi,  1,0 si,  0,0 st
MB mem :   9189,4 total,    404,1 free,   6637,8 used,   4756,9 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2551,6 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TEMPO+ COMANDO
   5378 vitorbe+  20   0   69,9g   3,1g   2,1g T   0,0  34,5 110:00.16 java
```

Aplicando SIGCONT

Depois de interromper os processos, vamos retomá-los utilizando o comando **kill -CONT + PID**. Com isso, o processo volta a funcionar normalmente.

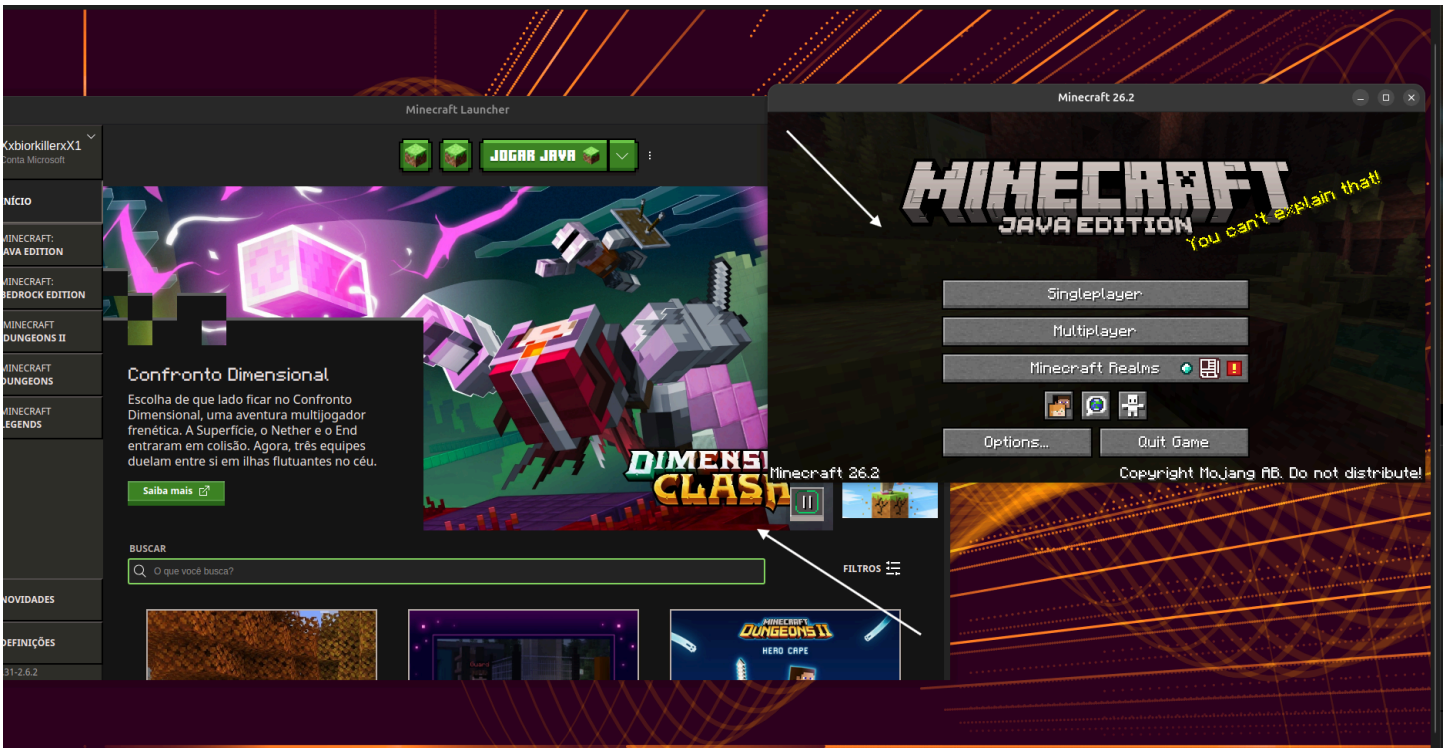
Para confirmar se os processos voltaram ao funcionamento normal, vamos utilizar o comando **top -p + PID**, que permite acompanhar informações como o estado, o uso de CPU e o consumo de memória.

No nosso caso, utilizamos:

kill -CONT 5009
kill -CONT 5378

```
vitorbeckham@ubunto3:~$ kill -CONT 5009
vitorbeckham@ubunto3:~$ kill -CONT 5378
vitorbeckham@ubunto3:~$
```

Aqui podemos ver que tudo voltou ao normal. A mensagem de parada forçada não aparece mais, e as animações e os sons voltaram a funcionar normalmente. Para confirmar o estado dos processos, podemos utilizar os comandos **top -p PPID** e **top -p PID**, como mostrado abaixo.



top -p 5009

```
vitorbeckham@ubunto3:~$ top -p 5009

top - 03:38:23 up  1:55,  1 user,  load average: 1,50, 1,24, 1,63
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s):  5,0 us,  4,6 sy,  0,0 ni, 89,2 id,  0,1 wa,  0,0 hi,  1,1 si,  0,0 st
MB mem :   9189,4 total,    467,8 free,   6571,2 used,   4633,5 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2618,3 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5009 vitorbe+  20   0 4256460 464508 245732 S   2,0   4,9   2:17.21 minecraft-launc
```

top -p 5378

```
vitorbeckham@ubunto3:~$ top -p 5378

top - 03:39:04 up  1:56,  1 user,  load average: 0,97, 1,14, 1,58
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s):  5,8 us,  5,1 sy,  0,0 ni, 87,9 id,  0,0 wa,  0,0 hi,  1,1 si,  0,0 st
MB mem :   9189,4 total,    434,3 free,   6602,3 used,   4665,0 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2587,1 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5378 vitorbe+  20   0   69,9g   3,1g   2,1g S  24,6  34,5 110:38.24 java
```

Alterando a prioridade

Depois disso, vamos alterar a prioridade dos processos. Inicialmente, a prioridade observada era **0**. Vamos alterá-la para **1top** e verificar o que acontece.

Para realizar essa alteração, utilizamos o comando **renice**, seguido do valor da nova prioridade, da opção **-p** e do PID do processo.

No nosso exemplo:

renice 1 -p 5009
renice 1 -p 5378

```
vitorbeckham@ubunto3:~$ renice 1 -p 5009
5009 (process ID) com prioridade antiga 0, prioridade nova 1
vitorbeckham@ubunto3:~$ renice 1 -p 5378
5378 (process ID) com prioridade antiga 0, prioridade nova 1
vitorbeckham@ubunto3:~$
```

Após realizar a alteração, utilizamos novamente o comando **top** para verificar como ficaram os processos e confirmar a nova prioridade.

```
vitorbeckham@ubunto3:~$ top -p 5009

top - 03:51:44 up  2:09,  1 user,  load average: 2,31, 2,46, 2,11
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s): 15,3 us, 14,1 sy,  0,1 ni, 67,2 id,  0,1 wa,  0,0 hi,  3,3 si,  0,0 st
MB mem :   9189,4 total,    549,9 free,   6462,3 used,   4792,9 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2727,2 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5009 vitorbe+  21   1 4407120 464712 245896 S   3,0   4,9   2:36.19 minecraft-launc
```

```
vitorbeckham@ubunto3:~$ top -p 5378

top - 03:52:23 up  2:09,  1 user,  load average: 1,92, 2,31, 2,08
Tarefas:   1 total,   0 em exec.,   1 dormindo,   0 parado,   0 zumbi
%CPU(s): 13,3 us,  9,0 sy,  0,0 ni, 75,6 id,  0,0 wa,  0,0 hi,  2,1 si,  0,0 st
MB mem :   9189,4 total,    552,1 free,   6461,7 used,   4797,7 buff/cache
MB swap:      0,0 total,      0,0 free,      0,0 used.   2727,7 avail mem

    PID USUARIO   PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPO+ COMANDO
   5378 vitorbe+  21   1   69,7g   2,9g   2,1g S  90,4  32,4 120:39.30 java
```

Aplicando SIGTERM

Por fim, vamos encerrar os processos utilizando o sinal **SIGTERM**. Esse sinal solicita que o processo seja encerrado, finalizando assim nossa análise. Como podemos observar na imagem acima, os dois processos ainda estão sendo executados normalmente.

Agora vamos fazer um experimento: vamos deixar o **PID 5378 órfão**, encerrando o processo pai e observando se o processo filho continuará sendo executado mesmo sem seu processo pai como podemos ver na imagem abaixo .

Os comandos utilizados foram:

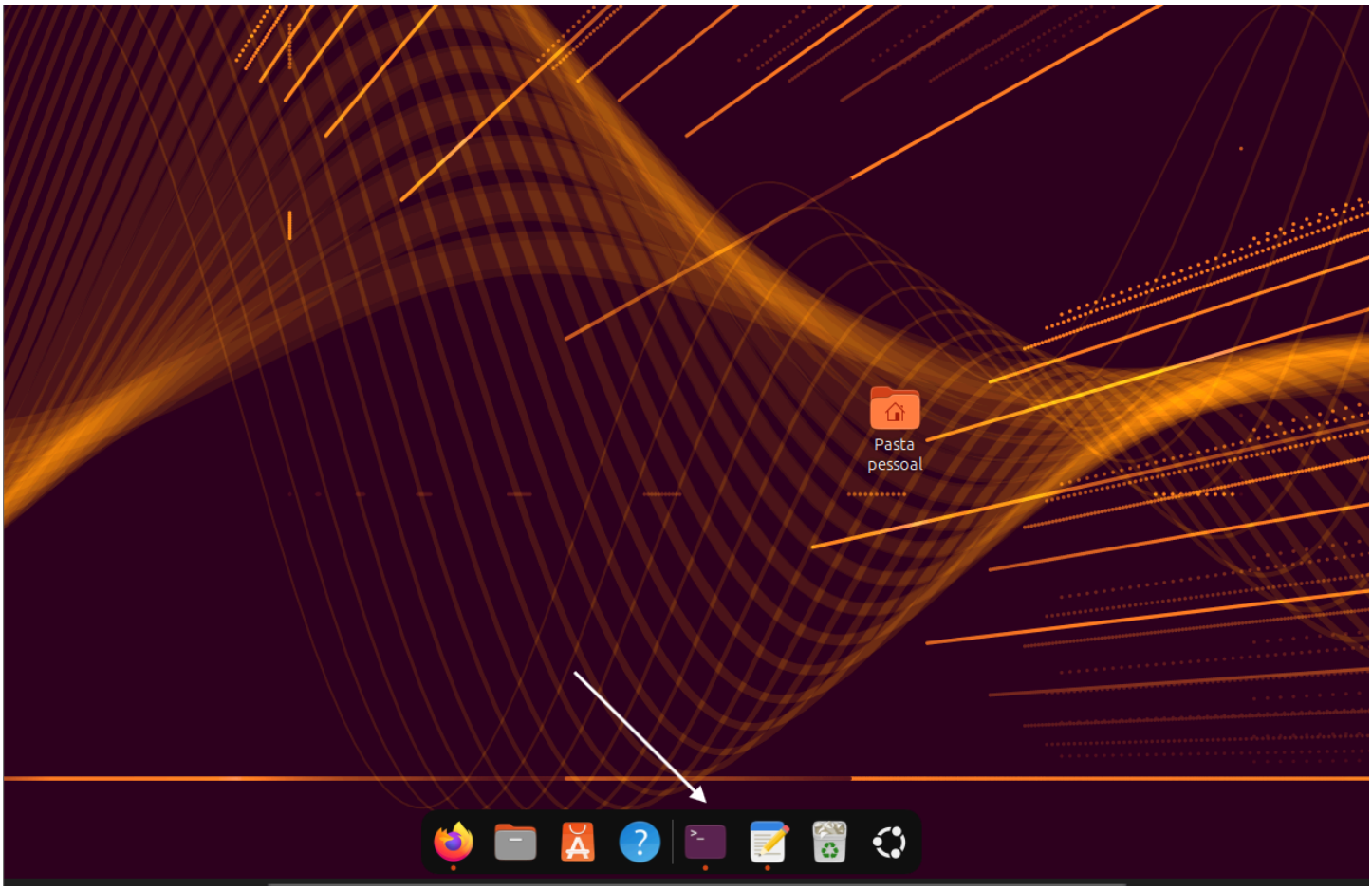
```
vitorbeckham@ubunto3:~$ kill -TERM 5009
vitorbeckham@ubunto3:~$ kill -TERM 5378
vitorbeckham@ubunto3:~$
```

kill -TERM 5009

Como podemos observar, o PID 5009 foi encerrado, deixando o PID 5378 órfão. Dessa forma, podemos verificar o que acontece com o processo filho após o encerramento do seu processo pai, neste experimento nao ocorreu mudançao o jogo continuou rodando normalmente.

kill -TERM 5378

Por fim, vamos encerrar o processo, deixando apenas o terminal e os outros processos que ainda estavam em execução. Com isso, finalizamos os testes e concluímos o nosso trabalho.



Concluão
Com os testes realizados, conseguimos entender na prática o funcionamento e o gerenciamento dos processos no sistema.

Conclusão

Durante a realização deste trabalho, conseguimos analisar na prática o funcionamento e o gerenciamento de processos no Linux utilizando o Minecraft como aplicação escolhida.
Começamos identificando os processos por meio dos comandos pgrep e pstree, o que permitiu encontrar os PID e entender a relação entre os processos pai e filho. A partir da árvore de processos, conseguimos definir os dois processos principais que seriam analisados durante a atividade: o processo do launcher e o processo responsável pela execução do jogo.
Depois disso, registramos informações importantes dos processos, como estado, prioridade, valor de nice, consumo de CPU e memória. Foi possível perceber que, apesar de os processos possuírem a mesma prioridade inicial, o consumo de recursos era diferente, principalmente no processo responsável pelo jogo, que apresentou um consumo maior de CPU e memória.
Também analisamos as threads e utilizamos o diretório /proc/PID para obter outras informações sobre os processos.
Na parte de manipulação dos processos, utilizamos os sinais **SIGSTOP** e **SIGCONT** para observar o que acontecia quando os processos eram interrompidos e depois retomados. Com o SIGSTOP, foi possível perceber na prática que o jogo e sua interface ficaram parados, enquanto com o

SIGCONT os processos voltaram a funcionar normalmente. Também alteramos a prioridade dos processos utilizando o comando renice e verificamos novamente suas informações por meio do top.

Por fim, utilizamos o **SIGTERM** para encerrar o processo pai e realizamos um teste para observar o comportamento do processo filho. Nesse experimento, o processo do jogo continuou funcionando mesmo após o encerramento do processo pai, permitindo observar na prática a situação de um processo órfão. Depois, encerramos também o processo do jogo, finalizando os testes realizados.

Com todas essas etapas, conseguimos relacionar os conceitos estudados em gerenciamento de processos com situações que realmente aconteceram durante a execução da aplicação. O trabalho permitiu entender melhor conceitos como **PID, PPID, processos pai e filho, threads, estado, prioridade, uso de CPU e memória e sinais de controle**, mostrando como essas informações podem ser utilizadas para acompanhar, diagnosticar e manipular processos em um sistema Linux.